

# Two-Stage Incremental Entity Resolution

FAISS blocking + Splink matching — online, per-query

**One system, three conversations**

Denis Robert

# How to use this deck

| Part | Audience               | Focus                         | After the talk     |
|------|------------------------|-------------------------------|--------------------|
| 1    | C-suite                | Value, risk, cost             | Decision / review  |
| 2    | Engineering leadership | Architecture, design, roadmap | Carry it forward   |
| 3    | Line engineers         | Orientation, stack, hands-on  | <b>Go hands-on</b> |

Line engineers: this orients you. The whitepaper ( `docs/entity_resolution_whitepaper.pdf` ) is the source of record; the numbers here come from it.

# Part 1 · Executive Summary

The 60-second version

# The problem

Organisations hold **many records for the same real person**, and they disagree.

- Names differ (e.g., `R. Martinez` vs `Robert Martinez` ).
- Addresses change; emails and dates go missing (**30% missing** here).
- Duplicates fragment CRM, risk, and reporting.

Wrong answers are expensive: linking two *different* people is usually far worse than missing two records of the same person.

# The solution: two stages

Fast retrieval + explainable matching — incremental (online, per-query).

```
Query → [embed] → [FAISS top-k]
      → [trained scorer] → Match / No match
```

This is an **incremental** resolver: it matches one incoming record against a fixed reference on demand (API / streaming insert). It is **not** a batch ER engine — it scores top- `k` candidates per query rather than materialising and clustering all pairwise comparisons at once.

- **Blocking** (FAISS): cheaply finds the few most likely matches.
- **Linkage** (trained scorer): weighs the evidence per field and returns a probability, not just yes/no. Splink trains the `m/u`; a lightweight scorer serves them per query.

# What it delivers

- **An incremental (online) matcher:** resolve one query record against a fixed reference in real time — the per-request path for an API or streaming insert. Not a batch-deduplication tool.
- **99.1% F1** on the synthetic test corpus (40,013 labelled queries: identical + six clerical perturbations + close + unrelated).
- **~32 ms/query amortized *batched* throughput;** the honest **online** per-query path is **~45 ms median** (embedding + FAISS + a lightweight trained scorer — no per-query Splink construction).
- **Persisted and reusable:** the index reloads without re-embedding the population.
- **Defensible numbers:** every result reproduces by re-running the code.

# Set the risk dial, not a magic number

Splink returns a **probability**; a pair is called a match when that probability is at or above a threshold  $\tau$ . One number encodes the business trade-off.

- **Raise**  $\tau$  → fewer false positives (fewer *wrong* links) at slightly more false negatives.
- **Lower**  $\tau$  → catch more true matches, at more false positives.

The cost-based rule:

$$\tau^* = C_{FP} / (C_{FP} + C_{FN})$$

If a **false link** (FP) is 20× more costly than a **missed link** (FN), then

$\tau^* = 20/21 \approx 0.95$  — the model is tuned to the cost of each mistake. (Valid under calibrated probabilities; with labels, sweep  $\tau$  on validation — whitepaper Appendix — Deriving  $\tau$ .)

One parameter lets a risk-averse line (fraud/AML) and a recall-first line (marketing) share the same model, each tuned to its own economics.

# Cost and scale

- The 50,000-record index is **~87 MB** today.
- Capacity is predictable from RAM;

| RAM    | Recommended capacity |
|--------|----------------------|
| 16 GB  | ~6.9 M records       |
| 32 GB  | ~13.8 M records      |
| 128 GB | ~55.1 M records      |

Around **7–14 M records** (or when you need replicated, multi-region durability) buy a **vector database**; before that the in-process index is the lean choice.



# The honest caveats

- Results are measured on **synthetic data**; production needs a **labelled sample of true duplicates** to calibrate and validate.
- Calibrate **m/u** with labels and tune the prior — supervised calibration improves precision on the de-duplicated 50k reference (→100%); keep the prior anchored, since the calibration gain is deck-dependent (on duplicate-rich data the untrained defaults can win).
- Volume must clear our capacity thresholds before any external-infra spend.

# The ask

- Sponsor a short **proof-on-real-data** phase: a representative, labelled sample of the population we must link (with access/privacy).
- We then commit to an F1 on *that* data, not the synthetic one.

**Value in one line:** incremental, online identity resolution — match one query record against a fixed reference at ~45 ms per query — with near-perfect, explainable P/R/F1; the per-query Splink rebuild is gone, replaced by a lightweight trained scorer (whitepaper §3.3). For one-off batch linkage of two large populations, use a dedicated batch linker instead.

# Part 2 · Engineering Leadership

Architecture, measurements, where to invest

# Architecture at a glance

Query  $\rightarrow$  MiniLM embed  $\rightarrow$  FAISS top-k  $\rightarrow$  lightweight scorer  $\rightarrow$   $p(M \mid \gamma)$

▲  
name / DOB / email / address comparisons,  $\tau$  threshold

- Embeddings are **L2-normalized**, so FAISS inner product = cosine.
- **Fellegi–Sunter**: per-field weight  $w = \log_2(m/u)$ , combined with a match prior, compared to  $\tau$ .
- **Train with Splink, infer with a lightweight scorer**: Splink trains the  $m/u$  once; each query is scored by a small weight-table scorer (Splink's comparison SQL, precompiled) — no per-query Splink **Linker** or DuckDB pipeline. The scorer matches Splink's posteriors to  $\sim 1e-7$ .
- A store abstraction lets us swap the in-memory index for an **external vector DB** later without changing blocking or linkage.

# Where performance comes from

Blocking recall is the ceiling; the scorer is where F1 is won or lost.

- On the confusion-matrix query set (40k queries): top-20 blocking recall **99.61%** → end-to-end recall **98.29%**. The binding stage depends on error kind: of 600 false negatives, 465 were retrieved-then-rejected by the matcher (mostly **initialled-first-name** positives: 99.7% blocked but only 90.5% linked), while only 135 were blocking misses (close-variant positives are blocker-capped at 97.7%).
- Per-kind through-pipeline recall is now recorded (initial-first-name 90.5%, close 97.6%, identity-typo 99.8%, everything else  $\geq 99.9\%$ ): the harder name-level noise is where the residual errors live.
- (Section 7, same perturbed deck, strategy comparison: default blocking recall 99.61%; compact raises it to 99.76% and F1 99.104% → 99.175%.)

Design implication: for close-variant noise, improving blocking recall is the lever; for initialled-first-name noise, it is the matcher / a stronger embedder at rank 1.

# The measured levers

| Lever             | Effect                                                                                                                                         | Verdict                                              |
|-------------------|------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------|
| Threshold $\tau$  | 0.85→0.95: F1 barely moves (99.104→99.116%); recall scarce                                                                                     | Small—retrieval/matcher is the lever, per error kind |
| Serialization     | single seed: R@20 99.76%, F1 99.175% compact vs 99.61%/99.104% default; five-seed companion confirms the ordering                              | Yes—small, reproducible                              |
| Blocking size $k$ | 20→100: +recall, scorer time only $\sim 1.10\times$ (23.9→26.2 s)                                                                              | Diminishing returns                                  |
| Weaken address    | no effect: F1 99.64% at every strength (reduced-scale 1,200-record sweep)                                                                      | No (this data)                                       |
| Retrain m/u       | §8.1: supervised <b>improves</b> (98.79 vs 98.05, precision→99.99%); §8.2 100k (perturbed deck): untrained 97.66 > supervised 97.43 > EM 94.86 | Precision-vs-recall: depends on the deck             |

## $\tau$ is the cost dial (engineering view)

Splink's decision rule: classify as *match* when  $P(M|\gamma) \geq \tau$  (range  $\sim[0.5, 0.999]$ ).

- **Raise**  $\tau$  → fewer false positives, at more false negatives.
- **Lower**  $\tau$  → more true matches, at more false positives.

Decision-theoretic setting:  $\tau^* = C_{FP} / (C_{FP} + C_{FN})$  — presumes a calibrated posterior, zero cost for correct decisions, and per-pair decisions (whitepaper Appendix — Deriving  $\tau$ ).

- Measured:  $\tau$  0.85 → 0.95 moved F1 barely (99.104% → 99.116%; FP 22→4) because recall — not threshold — is the scarce resource.
- With recall headroom, raising  $\tau$  is the cheapest precision lever — the mechanism to encode FP/FN business costs. (Methods: whitepaper **Appendix — Deriving  $\tau$** .)

# Calibration: deck-dependent, worth doing with real labels

Retraining the match model helps on some decks and hurts on others — the prior and the cost model decide.

1. **Supervised calibration improves results on the de-duplicated 50k reference (§8.1):** fitting `m/u` on labelled pairs lifts F1 to **98.79% vs 98.05% untrained**, precision to **99.99%** (recall 96.59→97.61%).
2. **On duplicate-rich data the untrained defaults win (§8.2 100k, perturbed deck):** untrained F1 97.66% > supervised 97.43% > EM 94.86% — calibrated schemes hit precision 100% but trade recall; the right variant depends on the deployment cost model (precision- vs recall-critical).

Supervised calibration uses genuine match evidence and helps on the near-duplicate-free reference; EM refits error terms to value-equality collisions and is the weakest on the duplicate-bearing benchmark.

**Rule:** the default config is a coherent bundle (weights + prior + `τ`). Change any part and re-validate **all three** on held-out data.



# Capacity and when to buy infra

- **Memory is the binding constraint** (~1,746 bytes/record).
- Move to an **external vector DB** around **7–14 M records**, or when you need shared/multi-region **durability and replication**, or beyond **~50 M** as a rule of thumb.
- The persisted index reloads **without re-embedding**, so redeploy and multi-consumer serving are cheap.

# Operational readiness

- Persisted, reload-friendly store with `update` / `delete` ; external-store interface is the vector-DB swap point.
- **Reproducible**: every experiment maps to a script; scripts accept `--input` .

## Roadmap

1. Invest in **linkage calibration** (labelled pairs + joint  $\tau$ /prior tuning).
2. Spend on **retrieval/embedding only where rank-1 / blocking recall is the binding constraint** (e.g. the initialled-first-name positives and close-variant ceiling here); canopy/embedding-blocked `m/u` training is a §Further Research option, not needed at this scale.
3. Add a **CI gate** locking F1 on a held-out labelled set.
4. **Gate go/no-go on internal corporate customer data** — the NC-voter run is only a public-data sanity check (pre-deduplicated; no DOB/email), so the production decision uses the org's own labelled data.

# Part 3 · Line Engineers

Orientation — where the knobs are

# What the code does

```
Blocker    : embed query → FAISS top-k candidates (cosine)
Scorer     : Splink-trained weight tables → per-candidate match probability
            (Splink comparison SQL, precompiled; no per-query Splink Linker)
Store      : add / update / delete / save / load (no re-embed on load)
```

- Match probability = Bayesian combination of per-field weights  $w = \log_2(m/u)$ , a match prior, thresholded at  $\tau$ .
- Splink is the **training** engine (estimates  $m/u$  and the prior); the lightweight scorer serves them at query time and matches Splink's posteriors to  $\sim 1e-7$ .
- In-memory index today; `IndexingStrategy` / `VectorDatabase` is the seam for a vector DB later.

# Scorecard to remember

- Confusion matrix (5k refs / 40k queries, perturbed deck): **F1 99.10%**, recall 98.29%, precision 99.94% (strict per-row definition).
- Same-set blocking recall: **99.61%** @k=20 (Section 7, same deck, default strategy; compact 99.76%).
- **~32 ms/query amortized batched** (embed + block + scorer); **cold per-query online path median ~45 ms** on the 50k index. Stage timings are measured **independently** (embedding ~27 ms, FAISS ~29 ms, scorer ~15 ms — overlapping, so they do not sum to the ~45 ms end-to-end median).
- NC-voter (real schema, synthetic mutations): **F1 92–94%**; ~88% top-20 blocking recall on the real schema leaves the retrieval stage capping recall (the scorer adds essentially no false positives as **k** grows).

Baseline engineering numbers — config, hardware, and data shape all move them.

# Reproducing experiments

Run from repo root; scripts have `--help` and deterministic seeds (default 42).

| Paper section                     | Script                                                           |
|-----------------------------------|------------------------------------------------------------------|
| §7 evaluation                     | <code>scripts/experiment_section7_eval.py</code>                 |
| §8 confusion matrix               | <code>scripts/experiment_confusion_matrix.py</code>              |
| §8.1 m/u calibration              | <code>scripts/experiment_mu_calibration.py</code>                |
| §8.2 duplicate benchmark          | <code>scripts/experiment_duplicate_benchmark.py</code>           |
| §8.3 F1 sweep                     | <code>scripts/experiment_f1_sweep.py</code>                      |
| calibration robustness            | <code>scripts/experiment_mu_tau_interaction.py</code>            |
| joint $\tau \times$ prior surface | <code>scripts/experiment_mu_prior_tau_surface.py</code>          |
| NC-voter replication              | <code>scripts/ncvoter/*</code> + <code>extract_ncvoter.py</code> |

For **your** data: population-based scripts accept `--input-records FILE`.

# Gotchas — tuning & thresholds

- **Calibrate  $m/u$  with labels — and tune the prior.** Supervised calibration improves results (98.79 vs 98.05 on the 50k perturbed deck, precision→99.99%); on the duplicate-bearing 100k data the ranking flips (untrained 97.66 > supervised 97.43 > EM 94.86), so re-validate prior +  $\tau$  + weights together against the deployment cost model (Appendix — Tuning the Match Prior).
- **$\tau$  encodes business cost.**  $\tau^* = C_{FP} / (C_{FP} + C_{FN})$  ; exposed via `--threshold` / `--tau` and the F1 sweep. Raise  $\tau$  for fewer wrong links, lower it for recall-first (Appendix — Deriving  $\tau$ ).
- **Tune the prior properly.** Anchor  $\lambda$  , fit with the prior pinned, tune  $\tau$  jointly, watch the score-shift guard (Appendix — Tuning the Match Prior).

# Gotchas — blocking & data

- **Blocking recall is a hard ceiling.** If recall is low, raise `k` or serialization, not the embedding model.
- **Address weakening didn't help** here — test, don't assume.
- **Real data needs a mutation/duplicate model** to score (`scripts/ncvoter/ncvoter_util.py`).
- **NC voter has no full DOB and no email** — those comparisons are weak there.



# Where the details live

Whitepaper: `docs/entity_resolution_whitepaper.pdf` (source: `.tex`).

- **§2 System Objective & Data Model** — the data model and a **central architecture diagram** (Fig. 1) showing the offline build, the per-query `embed→block→score→decide` path, and the trained weights feeding the matcher.
- **§3 Two-Stage** — blocking/linkage contract and costs; **Address volatility & temporal decay** (§3.5) consolidates why address evidence changes, the continuous retrieval-time decay and the bucketed comparison-level alternative, and their measured impact.
- **§7 Evaluation** — recall, thresholds, latency, ablations.
- **§8, 8.1, 8.2, 8.3** — headline results and the "don't retune alone" evidence.
- **§9 NC-voter** — real schema, mutation model, `k` scaling.
- **Appendix: Deriving  $\tau$**  — cost / F1 / GMM / transitivity methods.
- **Appendix: Tuning the Match Prior** — anchor  `$\lambda$` , then tune `( $\lambda$ ,  $\tau$ )` jointly (keep the prior coherent across retraining).
- **Use of AI** — disclosure and verification.

# Checklist before you leave

1. Run `scripts/experiment_confusion_matrix.py --count 5000` and read the JSON.
2. Exercise the calibration comparison: `scripts/experiment_mu_calibration.py --index-dir data --query-count 2000` (supervised improves on the 50k reference; the duplicate-bearing benchmark shows the untrained defaults can win — deck-dependent).
3. Swap `--input-records <your file>` into a population-based experiment.
4. Find where **blocking vs matcher** (per error kind) caps your F1 — then plan retrieval or linkage work accordingly.

That is the fastest way to make these results your own.

# Thank you

**FAISS + Splink — one pipeline, three audiences**

Bring a labelled sample of your real duplicates.